StockFlow

Sistema de gerenciamento de estoque

1. Objetivo

Desenvolver uma aplicação web utilizando Laravel e TypeScript para gerenciar o estoque de uma pequena loja. O sistema deverá permitir o cadastro de produtos, o registro de movimentações de estoque e o acompanhamento das quantidades disponíveis. O objetivo didático é praticar o desenvolvimento de aplicações web com Laravel, integração com banco de dados relacional, organização de código, validação de dados e interatividade no frontend com TypeScript.

2. Tecnologias obrigatórias

- PHP 8.4+
- Laravel 13
- MySQL
- HTML e CSS
- TypeScript
- Vite
- Git e GitHub

Utilizar Blade para construir as páginas e TypeScript vanilla para as funcionalidades interativas. Não React, Vue ou outro framework frontend. Os nomes de classes, métodos, propriedades, variáveis, tabelas e arquivos devem ser escritos em inglês. A interface apresentada ao usuário pode permanecer em português.

3. Funcionalidades

3.1. Dashboard

Criar uma página inicial com indicadores do estoque:

- Quantidade de produtos cadastrados.
- Quantidade total de unidades disponíveis.
- Quantidade de produtos com estoque baixo.
- Últimas movimentações registradas.

3.2. Cadastro e gerenciamento de produtos

- Cadastrar produtos.
- Listar produtos.
- Visualizar os detalhes de um produto.
- Editar os dados de um produto.
- Pesquisar produtos pelo nome ou SKU.
- Filtrar produtos por categoria.
- Filtrar produtos com estoque baixo.

3.3. Cada produto deverá possuir:

- Nome.
- SKU (código único do produto).
- Categoria.
- Descrição (opcional).
- Quantidade mínima desejada em estoque.
- Quantidade atual em estoque.

Um produto recém-cadastrado deverá começar com estoque zero. A quantidade disponível não deverá ser alterada diretamente pelo formulário de edição. As

categorias poderão ser inseridas inicialmente por meio de seeders. Não é necessário implementar um módulo completo de gerenciamento de categorias.

3.4. Movimentações de estoque

O sistema deverá permitir registrar dois tipos de movimentação:

1. Entrada:

    - Selecionar um produto.
    - Informar a quantidade recebida.
    - Informar o motivo da entrada.

2. Saída:

    - Selecionar um produto.
    - Informar a quantidade retirada.
    - Informar o motivo da saída.

Toda movimentação deverá registrar:

- Produto relacionado.
- Tipo de movimentação (ENTRY ou EXIT).
- Quantidade movimentada.
- Motivo.
- Data e horário do registro.

O sistema deverá atualizar automaticamente a quantidade atual do produto após cada movimentação.

3.5. Histórico de movimentações

Permitir consultar o histórico de movimentações, mostrando:

- Produto.
- Tipo de movimentação.
- Quantidade.
- Motivo.

- Data e horário.

Também deverá ser possível consultar o histórico individual de um produto. As movimentações registradas não deverão ser editáveis ou excluídas nesta versão do sistema.

4. Regras de negócio

As seguintes regras deverão ser respeitadas:

1. O SKU deve ser único.

2. A quantidade de uma movimentação deve ser sempre maior que zero.

3. Não será permitido registrar uma saída superior à quantidade disponível.

4. O estoque nunca poderá ficar negativo.

5. Um produto será considerado com estoque baixo quando sua quantidade atual for menor ou igual à quantidade mínima definida.

6. Toda entrada ou saída deverá gerar um registro de movimentação.

7. O registro da movimentação e a atualização do estoque deverão ocorrer dentro de uma transação do banco de dados. Caso uma das operações falhe, ambas deverão ser revertidas.

8. A quantidade atual de estoque somente poderá ser alterada por meio de movimentações.

5. Utilização do TypeScript

O TypeScript deverá ser utilizado para implementar interações reais na interface. São obrigatórias pelo menos estas três funcionalidades:

1. Pesquisa dinâmica: permitir pesquisar produtos sem precisar recarregar toda a página, utilizando `fetch()` para consultar o backend.

2. Prévia de movimentação: ao selecionar o tipo de movimentação e informar uma quantidade, apresentar ao usuário como ficará o estoque após a operação.

3. Validação visual: alertar o usuário quando tentar registrar uma saída superior ao estoque disponível e solicitar confirmação antes de concluir uma movimentação.

A validação no frontend serve para melhorar a experiência do usuário. O Laravel deverá continuar sendo responsável por validar e garantir as regras de negócio.

6. Requisitos técnicos do Laravel

Utilizar os recursos do framework para:

- Organizar as rotas da aplicação.
- Implementar Controllers.
- Criar Models e relacionamentos com Eloquent.
- Criar migrations para as tabelas.
- Utilizar seeders para dados iniciais.
- Validar requisições.
- Utilizar proteção CSRF nos formulários.
- Utilizar route model binding quando apropriado.
- Implementar transações de banco de dados.
- Exibir mensagens de sucesso e erro.
- Organizar as views com Blade.

7. Estrutura de dados sugerida

A aplicação deverá possuir, no mínimo, três entidades:

- Category
- Product
- StockMovement

Relacionamentos esperados:

- Uma categoria possui vários produtos.
- Um produto pertence a uma categoria.
- Um produto possui várias movimentações.
- Uma movimentação pertence a um produto.

8. Interface

Desenvolver uma interface moderna, organizada e responsiva. O dashboard deverá apresentar os indicadores de forma visualmente agradável. As páginas de cadastro, listagem e movimentação deverão ser intuitivas, com mensagens claras para o usuário. Não é necessário criar um design extremamente elaborado. O foco está na funcionalidade e na qualidade da implementação.

9. Controle de versão e documentação

Utilizar Git desde o início do desenvolvimento.

- Criar commits pequenos e descritivos.
- Organizar o projeto em um repositório GitHub.
- Criar um README com descrição, tecnologias, funcionalidades e instruções de instalação.
- Não versionar credenciais ou informações sensíveis.

10. Fora do escopo

Para manter o exercício pequeno e objetivo, não implementar nesta versão:

- Cadastro de clientes.
- Vendas e pagamentos.
- Controle financeiro.
- Cadastro de fornecedores.
- Emissão de notas fiscais.
- Múltiplos estabelecimentos ou depósitos.
- Leitura de códigos de barras.
- Controle de permissões avançado.
- Aplicativo mobile.

Autenticação de usuários poderá ser implementada como uma evolução posterior, caso o núcleo da aplicação esteja concluído.

11. Critérios de conclusão

O exercício será considerado concluído quando:

- O usuário conseguir cadastrar e editar produtos.
- O sistema conseguir registrar entradas e saídas.
- O estoque for atualizado corretamente.
- Saídas superiores ao estoque disponível forem impedidas.
- O histórico de movimentações estiver disponível.
- O dashboard apresentar os indicadores corretamente.
- A pesquisa dinâmica funcionar com TypeScript.
- As principais validações forem realizadas no backend.
- A aplicação estiver organizada e documentada.